

Nome: Camila de Oliveira Gomes

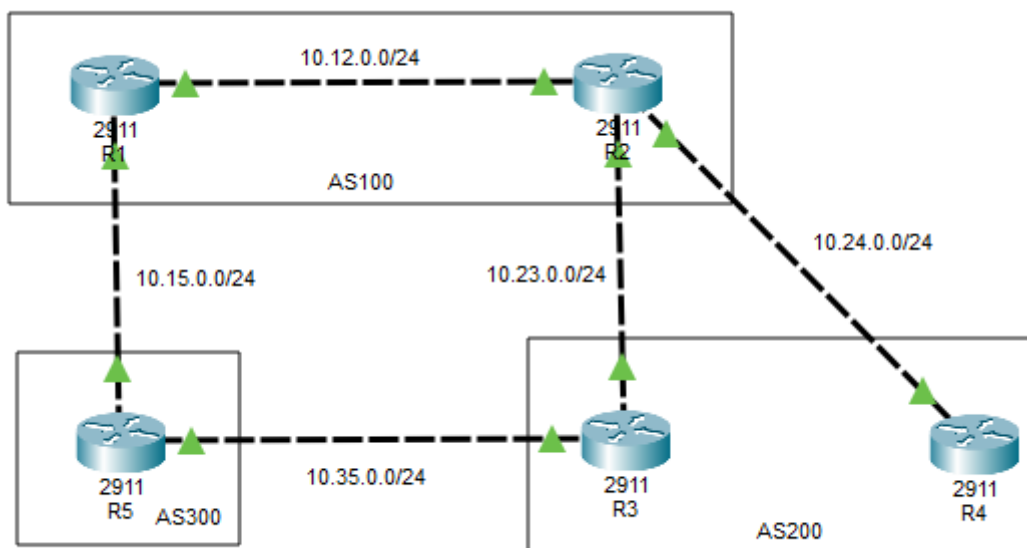
Trabalho GA

Este repositório contém a infraestrutura e os arquivos de configuração para emular uma topologia de rede distribuída em 3 Sistemas Autônomos (AS) utilizando roteadores virtuais baseados no daemon BIRD 2 e orquestrados via Docker Compose. O ambiente permite executar e comparar de forma reprodutível os protocolos RIPv2, OSPFv2 e eBGP, além de mensurar métricas empíricas de trânsito e sobrecarga de controle.

1. Topologia Física e Lógica

A rede é composta por 5 roteadores distribuídos da seguinte forma:

- AS 100: R1 e R2
- AS 200: R3 e R4
- AS 300: R5



1.1. Plano de Endereçamento IP e Interfaces

Enlace	Rede	Interface A (IP)	Interface B (IP)
R1 para R2	10.12.0.0/24	R1 eth0 (10.12.0.10)	R2 eth0 (10.12.0.20)
R1 para R5	10.15.0.0/24	R1 eth1 (10.15.0.10)	R5 eth0 (10.15.0.50)
R2 para R3	10.23.0.0/24	R2 eth1 (10.23.0.20)	R3 eth0 (10.23.0.30)
R2 para R4	10.24.0.0/24	R2 eth2 (10.24.0.20)	R4 eth0 (10.24.0.40)
R3 para R5	10.35.0.0/24	R3 eth1 (10.35.0.30)	R5 eth1 (10.35.0.50)

1.2. Tabela de Encaminhamento

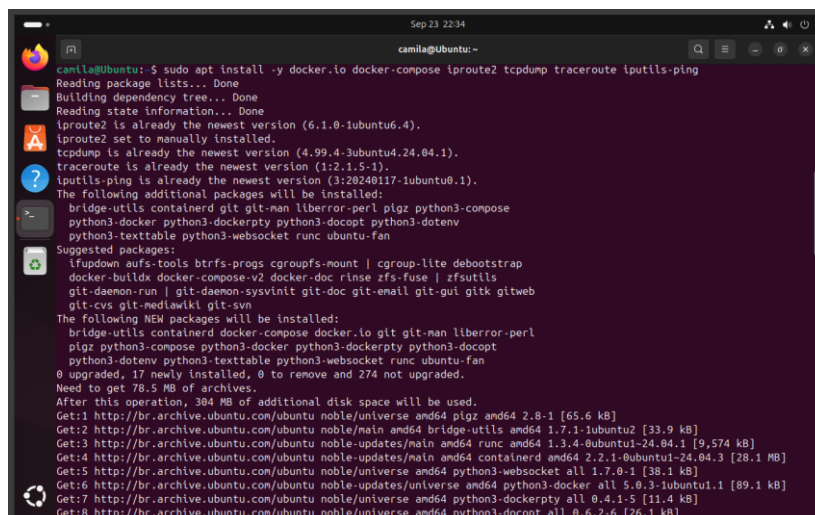
Rede de Destino	R1	R2	R3	R4	R5
10.12.0.0/24	--	--	R2	R2	R1
10.15.0.0/24	--	R1	R5	R2	--
10.23.0.0/24	R2	--	--	R2	R3
10.24.0.0/24	R2	--	R2	--	R1 ou R3
10.35.0.0/24	R5	R3	--	R2	--

2. Pré-requisitos

- Plataforma de Roteamento: <https://bird.network.cz/>
- Máquina Virtual: Oracle VirtualBox com Linux Ubuntu
- Docker Composer

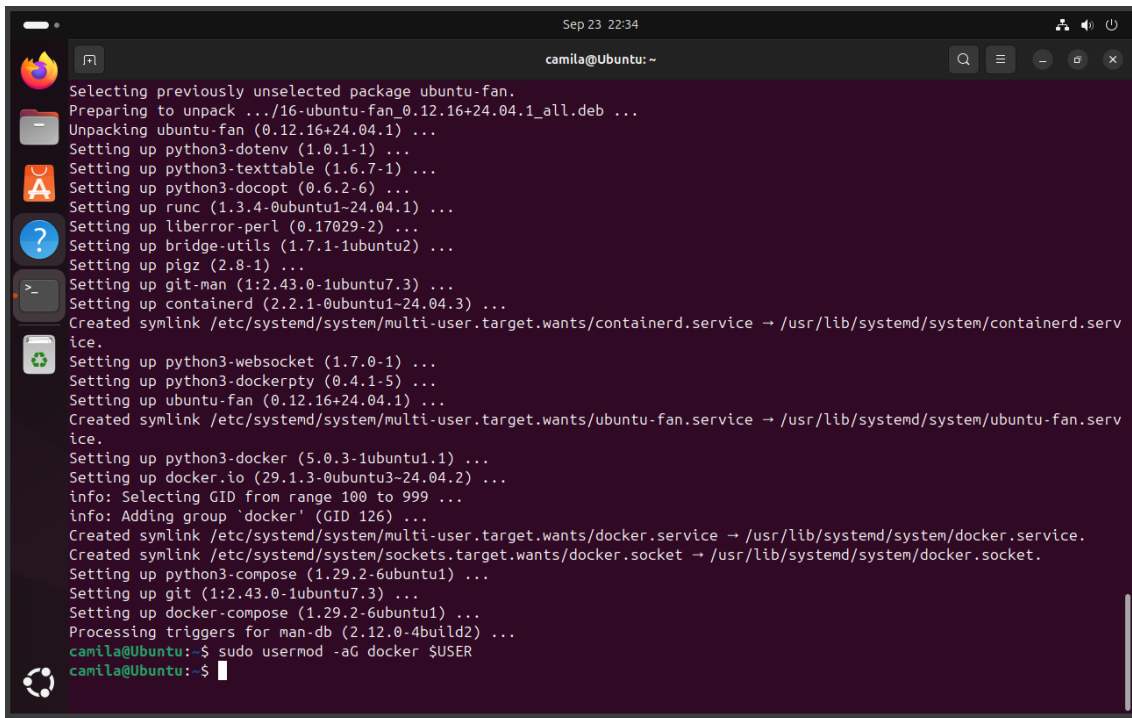
Para fazer a instalação do BIRD com o Docker, execute os comandos no prompt para instalar o BIRD, Docker, o utilitário Docker Composer e as ferramentas de diagnóstico:

- Atualizar lista de pacotes: `sudo apt update`
- Instalar Docker, Docker Compose, Git e ferramentas de rede: `sudo apt install -y docker.io docker-compose-v2 git traceroute iputils-ping tcpdump python3-matplotlib`



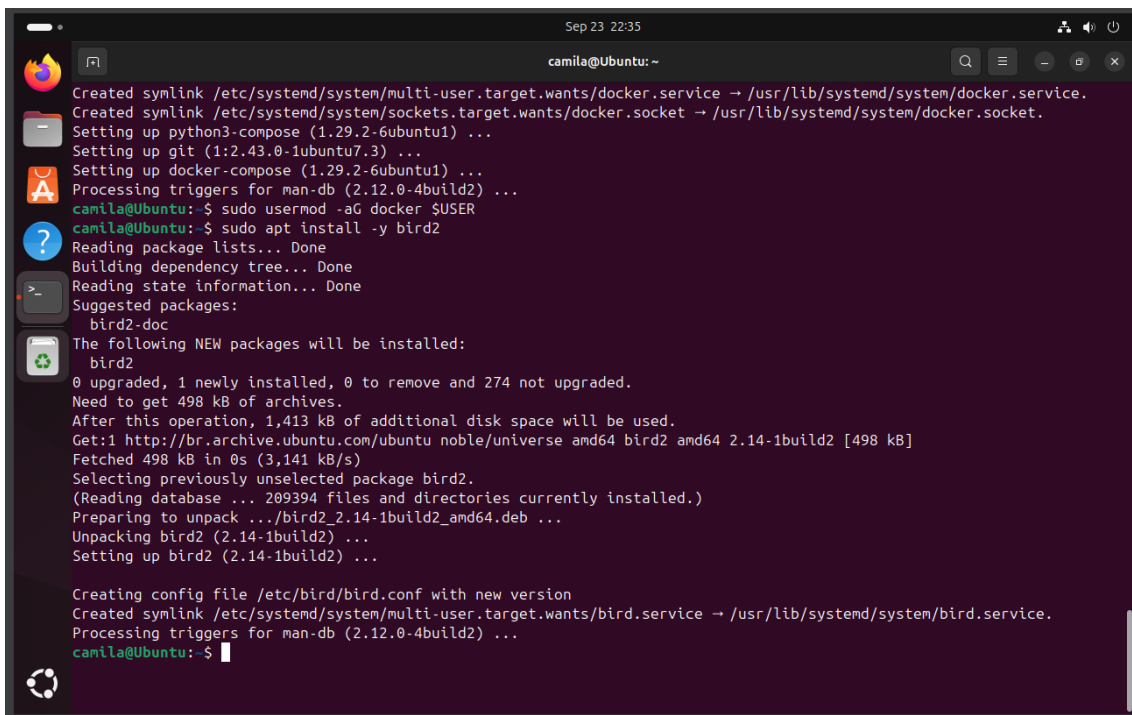
```
Sep 23 22:34
camila@Ubuntu: ~
camila@Ubuntu:~$ sudo apt install -y docker.io docker-compose iproute2 tcpdump traceroute iputils-ping
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
iproute2 is already the newest version (6.1.0-1ubuntu6.4).
iproute2 set to manually installed.
tcpdump is already the newest version (4.99.4-3ubuntu4.24.04.1).
traceroute is already the newest version (1:2.1.5-1).
iputils-ping is already the newest version (3:20240117-1ubuntu0.1).
The following additional packages will be installed:
  bridge-utils containerd git git-man liberror-perl pigz python3-compose
  python3-docker python3-dockerpty python3-dockerpty python3-dockerpty
  python3-dotenv python3-texttable python3-websocket runc ubuntu-fan
Suggested packages:
  ifupdown aufs-tools btrfs-progs cgroupfs-mount | cgroup-lite debotstrap
  docker-buildx docker-compose-v2 docker-doc rinse zfs-fuse | zfsutils
  git-daemon-run | git-daemon-sysvinit git-doc git-email git-gui gitk gitweb
  git-cvs git-mediawiki git-svn
The following NEW packages will be installed:
  bridge-utils containerd docker-compose docker.io git git-man liberror-perl
  pigz python3-compose python3-docker python3-dockerpty python3-dockerpty
  python3-dotenv python3-texttable python3-websocket runc ubuntu-fan
0 upgraded, 17 newly installed, 0 to remove and 274 not upgraded.
Need to get 78.5 MB of archives.
After this operation, 304 MB of additional disk space will be used.
Get:1 http://br.archive.ubuntu.com/ubuntu noble/universe amd64 pigz amd64 2.8-1 [65.6 kB]
Get:2 http://br.archive.ubuntu.com/ubuntu noble/main amd64 bridge-utils amd64 1.7.1-1ubuntu2 [33.9 kB]
Get:3 http://br.archive.ubuntu.com/ubuntu noble-updates/main amd64 runc amd64 1.3.4-0ubuntu1-24.04.1 [9,574 kB]
Get:4 http://br.archive.ubuntu.com/ubuntu noble-updates/main amd64 containerd amd64 2.2.1-0ubuntu1-24.04.3 [28.1 MB]
Get:5 http://br.archive.ubuntu.com/ubuntu noble/universe amd64 python3-websocket all 1.7.0-1 [38.1 kB]
Get:6 http://br.archive.ubuntu.com/ubuntu noble-updates/universe amd64 python3-docker all 5.0.3-1ubuntu1.1 [89.1 kB]
Get:7 http://br.archive.ubuntu.com/ubuntu noble/universe amd64 python3-dockerpty all 0.4.1-5 [11.4 kB]
Get:8 http://br.archive.ubuntu.com/ubuntu noble/universe amd64 python3-dockerpty all 0.6.2-6 [26.1 kB]
```

- Conceder permissão ao seu usuário para executar o Docker sem sudo (opcional): `sudo usermod -aG docker $USER`



```
Sep 23 22:34
camila@Ubuntu: ~
Selecting previously unselected package ubuntu-fan.
Preparing to unpack .../16-ubuntu-fan_0.12.16+24.04.1_all.deb ...
Unpacking ubuntu-fan (0.12.16+24.04.1) ...
Setting up python3-dotenv (1.0.1-1) ...
Setting up python3-texttable (1.6.7-1) ...
Setting up python3-docopt (0.6.2-6) ...
Setting up runc (1.3.4-0ubuntu1-24.04.1) ...
Setting up liberror-perl (0.17029-2) ...
Setting up bridge-utils (1.7.1-1ubuntu2) ...
Setting up pigz (2.8-1) ...
Setting up git-man (1:2.43.0-1ubuntu7.3) ...
Setting up containerd (2.2.1-0ubuntu1-24.04.3) ...
Created symlink /etc/systemd/system/multi-user.target.wants/containerd.service → /usr/lib/systemd/system/containerd.service.
Setting up python3-websocket (1.7.0-1) ...
Setting up python3-dockerpty (0.4.1-5) ...
Setting up ubuntu-fan (0.12.16+24.04.1) ...
Created symlink /etc/systemd/system/multi-user.target.wants/ubuntu-fan.service → /usr/lib/systemd/system/ubuntu-fan.service.
Setting up python3-docker (5.0.3-1ubuntu1.1) ...
Setting up docker.io (29.1.3-0ubuntu3-24.04.2) ...
info: Selecting GID from range 100 to 999 ...
info: Adding group 'docker' (GID 126) ...
Created symlink /etc/systemd/system/multi-user.target.wants/docker.service → /usr/lib/systemd/system/docker.service.
Created symlink /etc/systemd/system/sockets.target.wants/docker.socket → /usr/lib/systemd/system/docker.socket.
Setting up python3-compose (1.29.2-6ubuntu1) ...
Setting up git (1:2.43.0-1ubuntu7.3) ...
Setting up docker-compose (1.29.2-6ubuntu1) ...
Processing triggers for man-db (2.12.0-4build2) ...
camila@Ubuntu:~$ sudo usermod -aG docker $USER
camila@Ubuntu:~$
```

- Instalação do BIRD: `sudo apt install -y bird2`



```
Sep 23 22:35
camila@Ubuntu: ~
Created symlink /etc/systemd/system/multi-user.target.wants/docker.service → /usr/lib/systemd/system/docker.service.
Created symlink /etc/systemd/system/sockets.target.wants/docker.socket → /usr/lib/systemd/system/docker.socket.
Setting up python3-compose (1.29.2-6ubuntu1) ...
Setting up git (1:2.43.0-1ubuntu7.3) ...
Setting up docker-compose (1.29.2-6ubuntu1) ...
Processing triggers for man-db (2.12.0-4build2) ...
camila@Ubuntu:~$ sudo usermod -aG docker $USER
camila@Ubuntu:~$ sudo apt install -y bird2
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Suggested packages:
  bird2-doc
The following NEW packages will be installed:
  bird2
0 upgraded, 1 newly installed, 0 to remove and 274 not upgraded.
Need to get 498 kB of archives.
After this operation, 1,413 kB of additional disk space will be used.
Get:1 http://br.archive.ubuntu.com/ubuntu noble/universe amd64 bird2 amd64 2.14-1build2 [498 kB]
Fetched 498 kB in 0s (3,141 kB/s)
Selecting previously unselected package bird2.
(Reading database ... 209394 files and directories currently installed.)
Preparing to unpack .../bird2_2.14-1build2_amd64.deb ...
Unpacking bird2 (2.14-1build2) ...
Setting up bird2 (2.14-1build2) ...

Creating config file /etc/bird/bird.conf with new version
Created symlink /etc/systemd/system/multi-user.target.wants/bird.service → /usr/lib/systemd/system/bird.service.
Processing triggers for man-db (2.12.0-4build2) ...
camila@Ubuntu:~$
```

3. Estrutura de Diretórios do Projeto

Os arquivos estão organizados em:

- bird-router/
 - Dockerfile
 - docker-compose.yml
 - configs/
 - r1/
 - bird.conf
 - r2/
 - bird.conf
 - r3/
 - bird.conf
 - r4/
 - bird.conf
 - r5/
 - bird.conf
 - configs_ospf/
 - r1.conf
 - r2.conf
 - r3.conf
 - r4.conf
 - r5.conf
 - configs_bgp/
 - r1.conf
 - r2.conf
 - r3.conf
 - r4.conf
 - r5.conf
 - configs_rip/
 - r1.conf
 - r2.conf
 - r3.conf
 - r4.conf
 - r5.conf

3.1. Explicando um pouco sobre cada arquivo

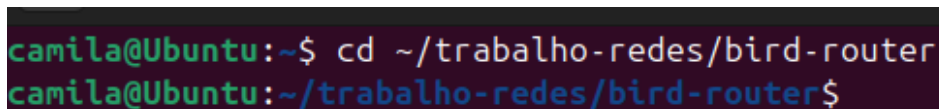
- Dockerfile: Define construção da imagem Linux dos roteadores virtuais baseada em Ubuntu. Ele instala o daemon de roteamento BIRD 2, as

ferramentas de diagnóstico e tráfego de rede (iproute2, ping, traceroute, tcpdump) e pré-cria o diretório /run/bird para estabilizar o socket de comunicação de controle (bird.ctl).

- **docker-compose.yml:** Cria e inicia simultaneamente os 5 contêineres (r1, r2, r3, r4 e r5), associa os endereços IP estáticos de cada interface (eth0, eth1, eth2) e define as 5 redes virtuais isoladas ponto-a-ponto (net_r1_r2, net_r1_r5 etc.), simulando os cabos físicos entre os roteadores.
- **configs/:** É a pasta onde ficam as configurações atualmente em execução nos contêineres.
- **bird.conf:** Dentro da subpasta de cada roteador é espelhado diretamente para o caminho /etc/bird/bird.conf do roteador virtual correspondente. O daemon BIRD lê exclusivamente esse arquivo para saber quais tabelas, protocolos e interfaces deve ativar no momento.
- **configs_ospf/:** Armazena as instruções do protocolo OSPFv2 para cada nó. Define a área, o custo métrico por enlace (cost 10) e a transmissão dos pacotes *Hello* via multicast (224.0.0.5).
- **configs_bgp/:** Armazena as instruções do protocolo eBGP. Define o número do Sistema Autônomo de cada roteador (as 100, as 200 ou as 300) e configura as sessões de emparelhamento TCP ponto-a-ponto com seus respectivos vizinhos.
- **configs_rip/:** Armazena as instruções do protocolo RIPv2. Configura o modo de envio e recebimento periódico de anúncios da tabela inteira via datagramas UDP em formato broadcast/multicast nas interfaces.

4. Utilizando os arquivos do GitHub

- Abra o terminal na sua máquina virtual e entre no diretório de trabalho: `cd ~/trabalho-redes/bird-router`



```
camila@Ubuntu:~$ cd ~/trabalho-redes/bird-router
camila@Ubuntu:~/trabalho-redes/bird-router$
```

4.1. OSPF

- Se desejar testar o OSPF, injetar as configurações do OSPFv2 nos roteadores: `for r in r1 r2 r3 r4 r5; do cp configs_ospf/$r.conf configs/$r/bird.conf; done && sudo docker compose restart`
`sleep 40`

```
camila@Ubuntu:~/trabalho-redes/bird-router$ for r in r1 r2 r3 r4 r5; do cp configs_ospf/$r.conf configs/$r/bird.conf; done && sudo docker compose restart && sleep 40
[sudo] password for camila:
[+] Restarting 5/5
✓ Container r4 Started 2.0s
✓ Container r5 Started 2.3s
✓ Container r2 Started 2.4s
✓ Container r1 Started 2.3s
✓ Container r3 Started 2.4s
```

4.2. BGP

Se desejar testar o BGP, injetar as configurações do eBGP nos roteadores: for r in r1 r2 r3 r4 r5; do cp configs_bgp/\$r.conf configs/\$r/bird.conf; done && sudo docker compose restart && sleep 5

```
camila@Ubuntu:~/trabalho-redes/bird-router$ for r in r1 r2 r3 r4 r5; do cp configs_bgp/$r.conf configs/$r/bird.conf; done && sudo docker compose restart && sleep 5
[+] Restarting 5/5
✓ Container r1 Started 2.7s
✓ Container r2 Started 2.7s
✓ Container r3 Started 2.1s
✓ Container r5 Started 2.6s
✓ Container r4 Started 2.3s
camila@Ubuntu:~/trabalho-redes/bird-router$
```

4.3. RIP

Se desejar testar o RIP, injetar as configurações do RIPv2 nos roteadores: for r in r1 r2 r3 r4 r5; do cp configs_rip/\$r.conf configs/\$r/bird.conf; done && sudo docker compose restart && sleep 5

```
camila@Ubuntu:~/trabalho-redes/bird-router$ for r in r1 r2 r3 r4 r5; do cp configs_rip/$r.conf configs/$r/bird.conf; done && sudo docker compose restart && sleep 5
[sudo] password for camila:
[+] Restarting 5/5
✓ Container r2 Started 8.4s
✓ Container r4 Started 6.9s
✓ Container r3 Started 8.3s
✓ Container r5 Started 8.2s
✓ Container r1 Started 8.4s
camila@Ubuntu:~/trabalho-redes/bird-router$
```

5. Coletas de Métricas

Para qualquer um dos cenários ativos, execute os seguintes passos no roteador de teste R1:

5.1. Cenário OSPF

- Tabela de rotas e vizinhos:
 - `sudo docker compose exec r1 birdc show route`
`sudo docker compose exec r1 birdc show ospf neighbors`

```
camila@Ubuntu:~/trabalho-redes/bird-router$ sudo docker compose exec r1 birdc show route
sudo docker compose exec r1 birdc show ospf neighbors
BIRD 2.14 ready.
Table master4:
10.35.0.0/24      unicast [ospf1 19:58:12.026] * I (150/20) [5.5.5.5]
    via 10.15.0.50 on eth1
10.12.0.0/24      unicast [direct1 19:57:25.933] * (240)
    dev eth0
    dev eth0      unicast [ospf1 19:58:12.026] I (150/10) [2.2.2.2]
10.15.0.0/24      unicast [direct1 19:57:25.933] * (240)
    dev eth1
    dev eth1      unicast [ospf1 19:58:06.053] I (150/10) [5.5.5.5]
10.23.0.0/24      unicast [ospf1 19:58:17.046] * I (150/20) [3.3.3.3]
    via 10.12.0.20 on eth0
10.24.0.0/24      unicast [ospf1 19:58:17.046] * I (150/20) [4.4.4.4]
    via 10.12.0.20 on eth0
BIRD 2.14 ready.
ospf1:
Router ID      Pri      State      DTime  Interface  Router IP
2.2.2.2        1        Full/DR    38.305  eth0        10.12.0.20
5.5.5.5        1        Full/DR    38.305  eth1        10.15.0.50
```

- Latência e RTT médio: `sudo docker compose exec r1 ping -c 10 10.24.0.40`

```
camila@Ubuntu:~/trabalho-redes/bird-router$ sudo docker compose exec r1 ping -c 10 10.24.0.40
PING 10.24.0.40 (10.24.0.40) 56(84) bytes of data.
64 bytes from 10.24.0.40: icmp_seq=1 ttl=63 time=0.066 ms
64 bytes from 10.24.0.40: icmp_seq=2 ttl=63 time=0.072 ms
64 bytes from 10.24.0.40: icmp_seq=3 ttl=63 time=0.279 ms
64 bytes from 10.24.0.40: icmp_seq=4 ttl=63 time=0.067 ms
64 bytes from 10.24.0.40: icmp_seq=5 ttl=63 time=0.077 ms
64 bytes from 10.24.0.40: icmp_seq=6 ttl=63 time=0.069 ms
64 bytes from 10.24.0.40: icmp_seq=7 ttl=63 time=0.075 ms
64 bytes from 10.24.0.40: icmp_seq=8 ttl=63 time=0.068 ms
64 bytes from 10.24.0.40: icmp_seq=9 ttl=63 time=0.068 ms
64 bytes from 10.24.0.40: icmp_seq=10 ttl=63 time=0.066 ms

--- 10.24.0.40 ping statistics ---
10 packets transmitted, 10 received, 0% packet loss, time 9250ms
rtt min/avg/max/mdev = 0.066/0.090/0.279/0.062 ms
```

- Captura de overhead (Hello de 48 bytes via proto 89): `sudo docker compose exec r1 timeout 30 tcpdump -i eth1 -n "proto 89"`

```

camila@Ubuntu:~/trabalho-redes/bird-router$ sudo docker compose exec r1 timeout 30 tcpdump -i eth1 -n "proto 89"
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on eth1, link-type EN10MB (Ethernet), snapshot length 262144 bytes
19:58:55.936372 IP 10.15.0.10 > 224.0.0.5: OSPFv2, Hello, length 48
19:58:55.964248 IP 10.15.0.50 > 224.0.0.5: OSPFv2, Hello, length 48
19:59:05.937054 IP 10.15.0.10 > 224.0.0.5: OSPFv2, Hello, length 48
19:59:05.962693 IP 10.15.0.50 > 224.0.0.5: OSPFv2, Hello, length 48
19:59:15.936149 IP 10.15.0.10 > 224.0.0.5: OSPFv2, Hello, length 48
19:59:15.963308 IP 10.15.0.50 > 224.0.0.5: OSPFv2, Hello, length 48

6 packets captured
6 packets received by filter
0 packets dropped by kernel

```

5.2. Cenário BGP

- Status das sessões (Established) e tabela de rotas BGP:

- `sudo docker compose exec r1 birdc show protocols`
`sudo docker compose exec r1 birdc show route`

```

camila@Ubuntu:~/trabalho-redes/bird-router$ sudo docker compose exec r1 birdc show protocols
sudo docker compose exec r1 birdc show route
BIRD 2.14 ready.
Name      Proto  Table  State  Since      Info
device1   Device ---    up     19:59:31.028
direct1   Direct ---    up     19:59:31.028
kernel1   Kernel master4 up     19:59:31.028
bgp_r2     BGP    ---    up     19:59:35.343 Established
bgp_r5     BGP    ---    up     19:59:35.756 Established
BIRD 2.14 ready.
Table master4:
10.35.0.0/24      unicast [bgp_r5 19:59:35.756] * (100) [AS300i]
                  via 10.15.0.50 on eth1
                  unreachable [bgp_r2 19:59:35.345 from 10.12.0.20] (100) [AS200i]
10.12.0.0/24      unicast [direct1 19:59:31.034] * (240)
                  dev eth0
                  via 10.12.0.20 on eth0
10.15.0.0/24      unicast [direct1 19:59:31.034] * (240)
                  dev eth1
                  via 10.15.0.50 on eth1
10.23.0.0/24      unicast [bgp_r2 19:59:35.345] * (100) [i]
                  via 10.12.0.20 on eth0
10.24.0.0/24      unicast [bgp_r2 19:59:35.345] * (100) [i]
                  via 10.12.0.20 on eth0

```

- Latência e RTT médio: `sudo docker compose exec r1 ping -c 10 10.24.0.40`

```

camila@Ubuntu:~/trabalho-redes/bird-router$ sudo docker compose exec r1 ping -c 10 10.24.0.40
PING 10.24.0.40 (10.24.0.40) 56(84) bytes of data.
64 bytes from 10.24.0.40: icmp_seq=1 ttl=63 time=0.123 ms
64 bytes from 10.24.0.40: icmp_seq=2 ttl=63 time=0.068 ms
64 bytes from 10.24.0.40: icmp_seq=3 ttl=63 time=0.100 ms
64 bytes from 10.24.0.40: icmp_seq=4 ttl=63 time=0.067 ms
64 bytes from 10.24.0.40: icmp_seq=5 ttl=63 time=0.075 ms
64 bytes from 10.24.0.40: icmp_seq=6 ttl=63 time=0.069 ms
64 bytes from 10.24.0.40: icmp_seq=7 ttl=63 time=0.077 ms
64 bytes from 10.24.0.40: icmp_seq=8 ttl=63 time=0.071 ms
64 bytes from 10.24.0.40: icmp_seq=9 ttl=63 time=0.068 ms
64 bytes from 10.24.0.40: icmp_seq=10 ttl=63 time=0.168 ms

--- 10.24.0.40 ping statistics ---
10 packets transmitted, 10 received, 0% packet loss, time 9362ms
rtt min/avg/max/mdev = 0.067/0.088/0.168/0.031 ms

```

- Captura de overhead (Keepalive de 19 bytes via TCP 179): `sudo docker compose exec r1 timeout 45 tcpdump -i eth1 -n "tcp port 179"`

```

camila@Ubuntu:~/trabalho-redes/bird-router$ sudo docker compose exec r1 timeout 45 tcpdump -i any -n "tcp port 179"
tcpdump: data link type LINUX_SLL2
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on any, link-type LINUX_SLL2 (Linux cooked v2), snapshot length 262144 bytes
20:00:35.573844 eth0 Out IP 10.12.0.10.48547 > 10.12.0.20.179: Flags [P.], seq 4270374249:4270374268, ack 3087715425, w
in 63, options [nop,nop,TS val 78664349 ecr 3030712070], length 19: BGP
20:00:35.573956 eth0 In IP 10.12.0.20.179 > 10.12.0.10.48547: Flags [.], ack 19, win 64, options [nop,nop,TS val 30307
71845 ecr 78664349], length 0
20:00:46.186202 eth1 Out IP 10.15.0.10.51001 > 10.15.0.50.179: Flags [P.], seq 2639943101:2639943120, ack 2935855057, w
in 63, options [nop,nop,TS val 3403448828 ecr 772152105], length 19: BGP
20:00:46.186304 eth1 In IP 10.15.0.50.179 > 10.15.0.10.51001: Flags [.], ack 19, win 64, options [nop,nop,TS val 77222
2491 ecr 3403448828], length 0
20:00:47.312498 eth1 In IP 10.15.0.50.179 > 10.15.0.10.51001: Flags [P.], seq 1:20, ack 19, win 64, options [nop,nop,T
S val 772223617 ecr 3403448828], length 19: BGP
20:00:47.352597 eth1 Out IP 10.15.0.10.51001 > 10.15.0.50.179: Flags [.], ack 20, win 63, options [nop,nop,TS val 34034
49995 ecr 772223617], length 0
20:00:48.956036 eth0 In IP 10.12.0.20.179 > 10.12.0.10.48547: Flags [P.], seq 1:20, ack 19, win 64, options [nop,nop,T
S val 3030785227 ecr 78664349], length 19: BGP
20:00:48.956136 eth0 Out IP 10.12.0.10.48547 > 10.12.0.20.179: Flags [.], ack 20, win 63, options [nop,nop,TS val 78677
731 ecr 3030785227], length 0

8 packets captured
8 packets received by filter
0 packets dropped by kernel

```

5.3. Cenário RIP

- Latência e RTT médio: `sudo docker compose exec r1 ping -c 10 10.24.0.40`

```

camila@Ubuntu:~/trabalho-redes/bird-router$ sudo docker compose exec r1 ping -c 10 10.24.0.40
PING 10.24.0.40 (10.24.0.40) 56(84) bytes of data.
64 bytes from 10.24.0.40: icmp_seq=1 ttl=63 time=0.126 ms
64 bytes from 10.24.0.40: icmp_seq=2 ttl=63 time=0.099 ms
64 bytes from 10.24.0.40: icmp_seq=3 ttl=63 time=0.072 ms
64 bytes from 10.24.0.40: icmp_seq=4 ttl=63 time=0.071 ms
64 bytes from 10.24.0.40: icmp_seq=5 ttl=63 time=0.073 ms
64 bytes from 10.24.0.40: icmp_seq=6 ttl=63 time=0.104 ms
64 bytes from 10.24.0.40: icmp_seq=7 ttl=63 time=0.082 ms
64 bytes from 10.24.0.40: icmp_seq=8 ttl=63 time=0.066 ms
64 bytes from 10.24.0.40: icmp_seq=9 ttl=63 time=0.083 ms
64 bytes from 10.24.0.40: icmp_seq=10 ttl=63 time=0.170 ms

--- 10.24.0.40 ping statistics ---
10 packets transmitted, 10 received, 0% packet loss, time 9243ms
rtt min/avg/max/mdev = 0.066/0.094/0.170/0.030 ms

```

- Captura de overhead (anúncios de 104 bytes via UDP 520): `sudo docker compose exec r1 timeout 30 tcpdump -i eth1 -n "port 520"`

```

camila@Ubuntu:~/trabalho-redes/bird-router$ sudo docker compose exec r1 timeout 30 tcpdump -i eth1 -n "port 520"
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on eth1, link-type EN10MB (Ethernet), snapshot length 262144 bytes
20:01:50.859767 IP 10.15.0.10.520 > 10.15.0.255.520: RIPv2, Response, length: 104
20:02:02.605194 IP 10.15.0.50.520 > 10.15.0.255.520: RIPv2, Response, length: 104

2 packets captured
2 packets received by filter
0 packets dropped by kernel

```

6. Comparação dos resultados

A avaliação experimental dos protocolos RIPv2, OSPFv2 e eBGP na topologia distribuída em três Sistemas Autônomos permitiu contrastar empiricamente as quatro métricas fundamentais de desempenho, controle e ocupação de recursos de rede.

6.1. Tabela Comparativa Consolidada

Métricas	RIPv2	OSPFv2	eBGP
Tamanho da Tabela de Roteamento	5 prefixos IPv4 (3 rotas rip1 + 2 locais)	5 prefixos IPv4 (3 rotas ospf1 + 2 locais)	5 prefixos IPv4 (3 rotas BGP + 2 locais)
Qtd. de Pacotes de Roteamento Enviados	2 pacotes periódicos em 30s	6 pacotes Hello em 30s	4 pacotes em 45s (Keepalive/ACK)
Taxa de Transmissão Utilizada	6,93 Bytes/s	9,60 Bytes/s	1,69 Bytes/s
Delay	0.070 ms	0.086 ms	0.082 ms

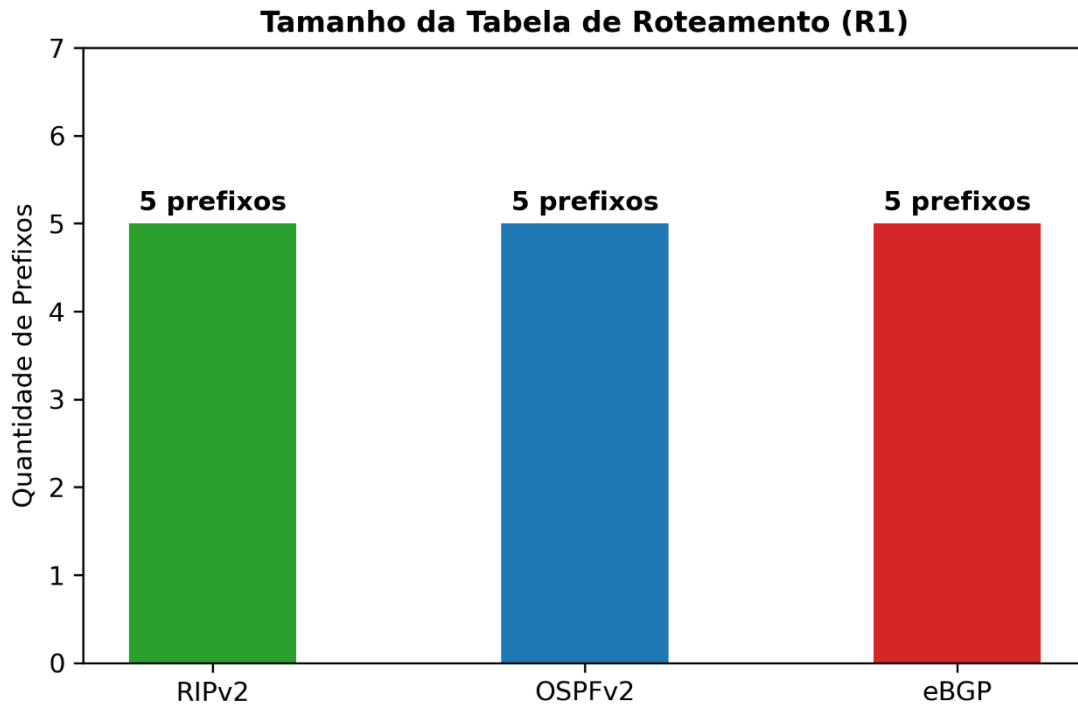
6.2. Análise Detalhada por Métrica e Gráficos

6.2.1. Tamanho da Tabela de Roteamento

Após a convergência da rede, o roteador R1 instalou exatamente 5 prefixos de rede IPv4 nos três protocolos avaliados, 2 redes locais diretamente conectadas (10.12.0.0/24 e 10.15.0.0/24) e 3 redes remotas aprendidas dinamicamente (10.23.0.0/24, 10.24.0.0/24 e 10.35.0.0/24).

Embora o número total de entradas seja idêntico devido ao escopo controlado do laboratório, a estrutura interna das rotas reflete os algoritmos:

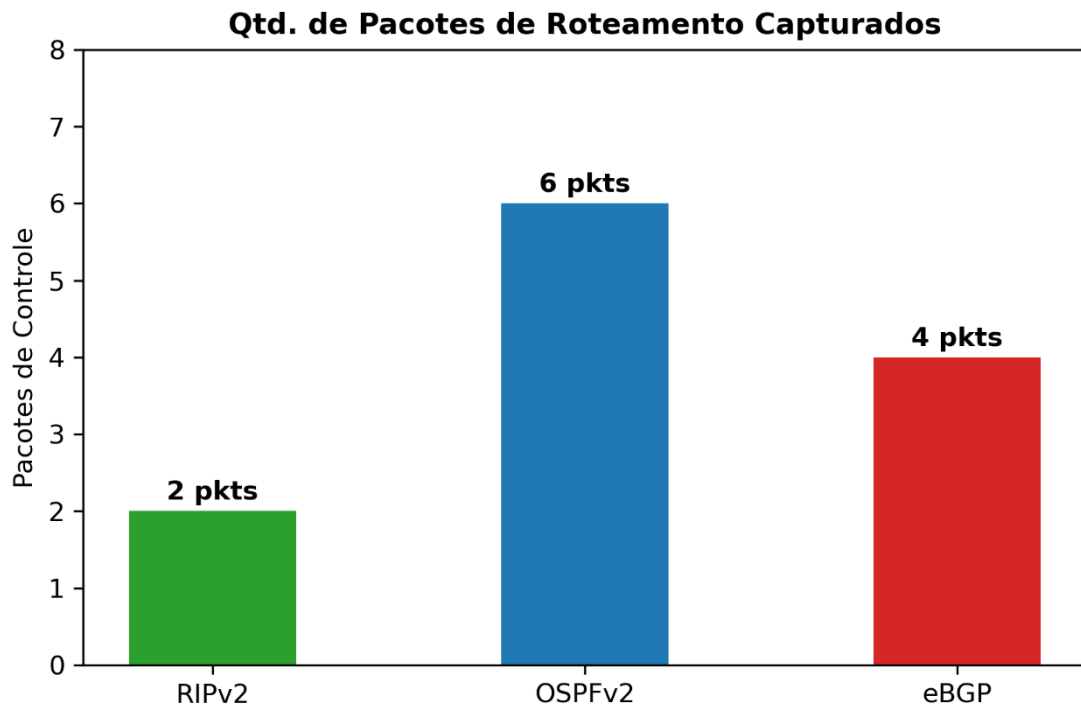
- RIPv2: Baseia-se no algoritmo de Bellman-Ford com distância administrativa 120 e custo fixo de 2 saltos (120/2).
- OSPFv2: Baseia-se no algoritmo de Dijkstra (SPF) com distância administrativa 150 e custos acumulados de enlaces (150/10 e 150/20).
- eBGP: Baseia-se em políticas de vetor de caminhos, enriquecendo as rotas com o histórico de trânsito dos Sistemas Autônomos ([AS300i]).



6.2.2. Quantidade de Pacotes de Roteamento Enviados

A análise do tráfego capturado com o utilitário tcpdump em regime estável demonstrou a frequência de troca de mensagens operacionais de cada protocolo:

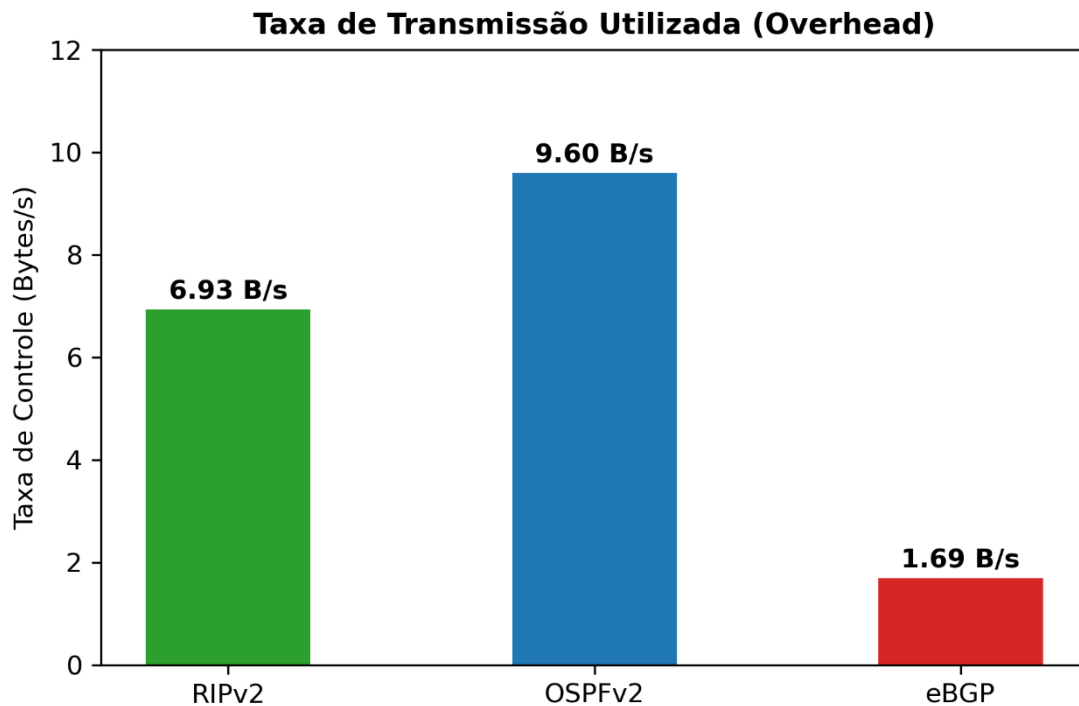
- RIPv2: Registrou 2 pacotes dentro de uma janela de 30 segundos, correspondendo à sua cadência padrão de difusão de atualizações completas periódicas não solicitadas na porta UDP 520.
- OSPFv2: Registrou 6 pacotes Hello de 48 Bytes em 30 segundos direcionados ao endereço de multicast 224.0.0.5. Essa transmissão bidirecional contínua (a cada 10 segundos por interface ativa) é necessária para a manutenção rápida da adjacência de vizinhança em estado Full/DR.
- eBGP: Registrou 4 pacotes em 45 segundos, consistindo unicamente em mensagens Keepalive e confirmações TCP ACK na porta 179.



6.2.3. Taxa de Transmissão Utilizada (Overhead de Controle)

O impacto real do protocolo no meio de transmissão foi calculado a partir do volume total de bytes de controle transmitidos pelo tempo de observação:

- OSPFv2: Apresentou a maior taxa de controle em regime estacionário (288 Bytes em 30s). Esse custo superior se justifica pela necessidade de monitoramento contínuo dos enlaces locais para detectar falhas e convergir em tempo sub-segundo.
- RIPv2: Gerou 208 Bytes em 30s devido ao tamanho fixo de cada anúncio UDP (104 Bytes). Em redes de grande porte, esse overhead escala consideravelmente por retransmitir a tabela integralmente a cada ciclo de anúncio.
- eBGP: Apresentou a menor sobrecarga de rede do comparativo (76 Bytes em 45s). Por operar em cima de uma sessão TCP estável e só enviar atualizações quando há mudanças reais na topologia (triggered updates), limita-se a pequenas mensagens de Keepalive de 19 Bytes em repouso.



6.2.4. Delay Médio Fim-a-Fim (RTT)

O teste de latência e perda de pacotes foi executado por meio de sequências de 10 pacotes ICMP disparados do nó R1 até o destino no nó R4 (10.24.0.40), passando por saltos intermediários:

- RIPv2: Latência média de 0.070 ms (min/avg/max/mdev = 0.063 / 0.070 / 0.079 / 0.005 ms) com 0% de perda.
- OSPFv2: Latência média de 0.086 ms (min/avg/max/mdev = 0.062 / 0.086 / 0.200 / 0.038 ms) com 0% de perda.
- eBGP: Latência média de 0.082 ms (min/avg/max/mdev = 0.066 / 0.082 / 0.147 / 0.023 ms) com 0% de perda.

Os resultados demonstram que, uma vez que as tabelas de encaminhamento estão devidamente instaladas no kernel do Linux, o plano de dados comuta os pacotes com velocidade praticamente idêntica e latência sub-milissegundo, governada principalmente pelo overhead de virtualização dos contêineres Docker locais.

